# FerreStock · Guía del proyecto

Documento interno para que los tres integrantes entendamos y podamos explicar todo el sistema.

| | | | |
|---|---|---|---|
| **Autores** | Ana Sofía Nagles García · María Mercedes Rincón Valencia · Juan David Camacho Tapiero | **Versión** | 1.0.0 (2026-10-15) |
| **Sistema en línea** | https://ferrestock-wtrq.onrender.com | **Repositorios** | GitHub `Ananagl/ferrestock-backend`, `-frontend`, `-docs` |
| **Taiga** | tree.taiga.io/project/ananagls-inventario_ferreteria_ia_2026 (Kanban; issues activos) | | |

## 1. Qué es FerreStock en una frase

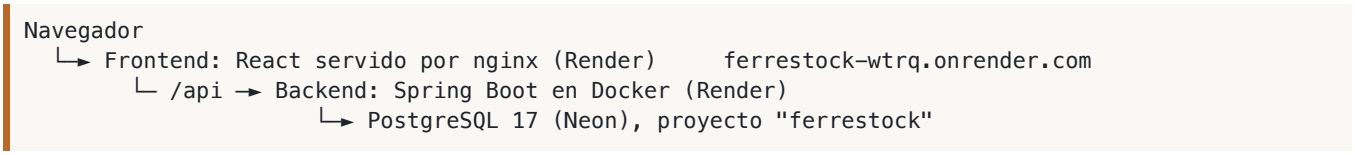
Es un sistema web de inventario para una ferretería. Registra productos, clientes y proveedores, compras, ventas con pago en efectivo, devoluciones y ajustes. Lleva el **kardex** y el **costo promedio ponderado (CPP)** y emite comprobantes simulados. Tiene dos roles: **Administrador** (todo) y **Operativo** (vende, compra y devuelve, pero no ve costos, ajustes, usuarios ni auditoría).

## 2. Cómo se construyó: IA utilizada y fases

**IA utilizada: Claude**, de Anthropic (modelo Claude Opus 5.5), a través de **Claude Code** en la app de escritorio. Funcionó como un equipo de ingeniería: redactó la documentación, propuso el diseño, programó backend y frontend, escribió y corrió las pruebas, configuró el CI/CD y el despliegue. **Nosotras/os definimos las decisiones y aprobamos cada fase** antes de avanzar. Los prompts usados están en `docs/prompts/FerreStock_prompts_de_ingenieria.pdf`.

| Fase | Qué se hizo | Dónde está |
|---|---|---|
| 1 · Análisis | Visión y alcance, requisitos RF/RNF, reglas de negocio, casos de uso, permisos por rol, escenarios de aceptación, riesgos y trazabilidad | `docs/fase-1-analisis` (9 documentos) |
| 2 · Diseño | Arquitectura, modelo de datos (ER), inventario y costos, API REST, seguridad, frontend, flujos críticos, entrega y 19 decisiones técnicas (ADR) | `docs/fase-2-diseno` |
| 3 · Preparación | Repositorios, base técnica del backend y del frontend, scripts de BD y verificación de encabezados | `docs/fase-3-preparacion` |
| 4 · Implementación | 11 incrementos: autenticación, usuarios, catálogo, terceros, inventario, ajustes, compras, ventas, devoluciones, comprobantes y auditoría | `docs/fase-4-implementacion` (un informe por incremento) |
| 5 · Pruebas | Unitarias, integración, seguridad, estrés, mutación, E2E, compatibilidad y carga; catálogo de 220 casos | `docs/fase-5-pruebas` |
| 6 · CI/CD y despliegue | GitHub Actions, reporte automático en Taiga, Docker, Render y Neon | `docs/fase-6-cicd` |
| 7 · Métricas y monitoreo | Métricas del equipo desde Taiga y monitoreo de errores con reporte automático | `docs/fase-7-metricas` |

## 3. Cómo está montado




- **Mismo origen:** el navegador solo habla con el frontend. nginx reenvía `/api` al backend, así las cookies de sesión y CSRF funcionan sin CORS.
- **Todo es gratis:** Render (2 servicios Docker) y Neon (PostgreSQL 17). En el plan gratis de Render el servicio "se duerme" tras 15 minutos sin uso: la primera visita tarda cerca de 1 minuto. **Abrirlo antes de presentar.**
- **Tres repositorios:** backend, frontend y docs. Ramas `main` (lo desplegable), `develop` (integración) y `feature/*` (una por incremento); commits en formato Conventional Commits en español.

## 4. Tecnologías y para qué sirve cada una

| Capa | Tecnología | Para qué la usamos |
|---|---|---|
| Backend | Java 21, Spring Boot 3.5, Maven | API REST `/api/v1/...`; Spring Web, Security, Data JPA, Validation, Actuator (salud) |
| | JPA/Hibernate + Lombok | Entidades y repositorios sin código repetitivo (`@Getter`, `@Builder`…) |
| | springdoc OpenAPI · Javadoc | Documentación de la API (Swagger solo en desarrollo) y del código |
| Base de datos | PostgreSQL 17 (Neon) · scripts SQL | Esquema con restricciones, triggers y datos iniciales en `db/01_esquema.sql` y `db/02_datos_iniciales.sql` |
| Frontend | React 19, Vite, Material UI 9 | Interfaz por módulos, tema claro/oscuro con **tokens de diseño**, responsive |
| | Formik + Yup · i18next · Axios | Formularios con validación igual a la del backend · español e inglés · cliente HTTP con CSRF e idioma |
| Pruebas | JUnit 5, Mockito, Spring Test, Testcontainers | Unitarias e integración contra un **PostgreSQL real** en contenedor |
| | Playwright + axe · Selenium · PIT · k6 | E2E y accesibilidad · compatibilidad (Edge/Chrome) · mutación · carga y estrés |
| Entrega | GitHub Actions · Docker · Render · Taiga | CI/CD; imagen de cada servicio; hosting; issues, tareas y métricas del equipo |

## 5. Backend: cómo está organizado

- **Paquetes por módulo** (`com.ferrestock.<modulo>`): autenticacion, usuarios, catalogo, terceros, inventario, ajustes, compras, ventas, devoluciones, comprobantes, auditoria y monitoreo, más `compartido` (errores, montos, seguridad) y `configuracion`.
- **Capas:** `Controller` (HTTP y permisos con `@PreAuthorize`) → `Service` (reglas de negocio y transacciones) → `Repository` (JPA). Los DTO de entrada se validan con anotaciones (`@NotBlank`, `@Digits`…) y mensajes traducidos.
- **Errores:** siempre con el mismo formato (ProblemDetail, RFC 9457): `codigo` estable (por ejemplo `EXISTENCIAS_INSUFICIENTES`), mensaje en el idioma del usuario y `errores` por campo, que el formulario muestra junto al campo. Un error inesperado devuelve 500 con un código de correlación y queda registrado.
- **Montos exactos:** `BigDecimal` con cantidades a 3 decimales, costos a 4 y dinero a 2 (HALF_UP). Viajan como texto en el JSON para no perder precisión.
- **Encabezado** en todos los archivos (nombre, descripción, fechas, autores, versión); `scripts/verificar_encabezados.sh` lo revisa en el CI.

### El corazón: inventario, costo promedio y concurrencia

- `MovimientoInventarioServicio` es **el único** que cambia existencias y CPP. Cada compra, venta, devolución, ajuste o anulación deja un movimiento en el **kardex** (con el saldo y el CPP resultantes). La base de datos impide editar o borrar el kardex con un trigger.
- **CPP** = (existencias × CPP actual + cantidad × costo de entrada) / (existencias + cantidad). Ejemplo: 10 a 1.000 + 30 a 1.400 = 40 unidades a **1.300**. Las salidas salen al CPP vigente.
- **Nunca existencias negativas**, ni con dos cajas vendiendo a la vez: los productos se bloquean (`SELECT … FOR UPDATE`) en orden de id dentro de la transacción. Además hay un `CHECK (existencias >= 0)` en la base. Probado: 2 ventas simultáneas de la última unidad confirman exactamente 1.
- **Consecutivos** (`VEN-000001`, `COM-`, `DEV-`, `AJU-`) en una tabla con bloqueo de fila: no se repiten y no dejan huecos.
- **Anulaciones (solo ADMIN, con motivo):** un documento no se borra, se anula con un movimiento compensatorio. Una compra no se anula si ya se vendió lo comprado o hubo entradas después; una venta no se anula si tiene devoluciones.

### Seguridad

| Mecanismo | Cómo funciona |
|---|---|

| | |
|---|---|
| Sesión de servidor | Cookie `FSSESSION` HttpOnly y Secure; las contraseñas se guardan con BCrypt |
| Sesión persistente | "Recordarme" 7 días con token persistente en BD (cookie `FSRECUERDAME`) |
| CSRF | Cookie `XSRF-TOKEN` + cabecera `X-XSRF-TOKEN` en cada POST/PUT/PATCH |
| Roles | `@PreAuthorize` en cada endpoint: el backend responde 403 aunque el frontend oculte la opción; el intento queda auditado |
| Cuentas | Bloqueo tras 5 intentos fallidos; contraseña temporal que se debe cambiar; si el ADMIN desactiva a alguien o le cambia el rol, su sesión se corta en la siguiente petición |
| Auditoría | Login, cambios de usuarios, productos, precios, documentos y accesos denegados; inmodificable (trigger) |

## 6. Frontend: cómo está organizado

- `src/modulos/<modulo>/` con `servicios` (llamadas a la API), `paginas`, `componentes` y `validacion` (esquemas Yup).
- `src/compartido/`: tabla paginada (tarjetas en móvil), campos de formulario, editor de líneas de documentos, diálogos de confirmación y anulación, formatos de COP y fechas.
- **Rutas protegidas** por sesión y rol; cada pantalla se carga solo cuando se visita.
- **Sesión persistente:** al recargar, la app consulta `/auth/sesion` y restaura el usuario.
- **Validación doble:** Yup en el navegador (respuesta inmediata) y Bean Validation en el servidor (seguridad). Los errores del servidor llegan al campo correcto.
- **i18n:** español e inglés, 14 namespaces; un script verifica que no falte ninguna traducción.
- **Accesibilidad:** axe-core en las E2E (WCAG A/AA), foco al primer error, etiquetas, contraste y navegación con teclado.
- **Tokens de diseño** y tema claro/oscuro; JSDoc en cada archivo.

## 7. Base de datos

- 14 tablas: usuario, categoria, producto, tercero, compra(+linea), venta(+linea), devolucion(+linea), ajuste, movimiento_inventario (kardex), consecutivo, auditoria, registro_error y persistent_logins.
- **Las reglas también viven en la base:** `CHECK` (existencias ≥ 0, cambio = recibido − total, motivo obligatorio si está anulada), `UNIQUE` (código de producto, documento del tercero, un producto por línea), claves foráneas y triggers de inmutabilidad.
- Hibernate solo **valida** el esquema (`ddl-auto=validate`); el esquema lo crean los scripts SQL. En local: `db/crear_bd.sh`. En la nube: `db/inicializar_remota.sh` (así se inicializó Neon).

## 8. Cómo se hicieron las pruebas

| Tipo | Qué se probó | Resultado real |
|---|---|---|
| Unitarias | Reglas de servicios con Mockito, cálculo del CPP, validador de NIT (módulo 11), contraseñas | 75 pasan |
| Integración / sistema | Cada endpoint contra PostgreSQL real (Testcontainers): contratos HTTP, errores, kardex, CPP y anulaciones | 86 pasan |
| Seguridad | 403 por rol, CSRF, bloqueo de cuenta, sesión cortada al desactivar, costos ocultos al operativo | incluidas en las 161 |
| Estrés y límite | 2 ventas simultáneas de 1 unidad → 1; 20 salidas sobre 10 → 10; 50 consecutivos distintos; decimales y ceros rechazados | pasan |
| Mutación (PIT) | Cambia el código a propósito y revisa si las pruebas lo detectan | 25 % (fuerza 83 %)* |
| E2E (Playwright) | Escenarios de aceptación en Chromium, Firefox y móvil, más axe | 216 pasan (matriz) |
| Compatibilidad | Selenium: ingreso, idioma, venta y sesión contra la app real | 3 pasan |
| Carga (k6) | Listados concurrentes, 200 ventas del mismo producto, picos | scripts listos** |

* PIT solo corre las unitarias; los servicios de documentos se prueban sobre todo con integración, por eso el porcentaje es bajo y la fuerza alta. ** k6 no estaba instalado en el equipo; se corren con `k6 run` (ver `pruebas-carga/README.md`). El catálogo completo (220 casos) está en `docs/fase-5-pruebas/01_catalogo_casos.md`.

```
./mvnw verify                     # backend completo (Docker abierto para Testcontainers)
npm test && npx playwright test   # frontend: lint, i18n y E2E
scripts/compilar_y_probar.sh      # todo el sistema
```

## 9. CI/CD y Taiga

```
push o pull request a develop / main  ─►  GitHub Actions
  ├─ backend:  encabezados → unitarias + integración (PostgreSQL) → cobertura
  │            en main además: mutación → imagen Docker → despliegue → salud
  ├─ frontend: lint → i18n → build → Playwright (3 navegadores)
  │            en main además: despliegue → Selenium en Edge
  └─ reportar (siempre):
       ¿falló algo? ─ sí ─► issue + tarea en Taiga para el autor del commit (aviso por correo)
                      └ no ─► cierra el issue abierto de esa etapa, si lo había
```

- **Despliega solo si todo pasa:** Render tiene el auto-deploy apagado. El job `desplegar` llama al deploy hook de Render cuando todas las etapas anteriores están en verde; sin hook, se despliega con **Manual Deploy** en Render.
- **Taiga:** `scripts/ci/taiga_reportar.sh` se autentica con los secretos `TAIGA_USUARIO` y `TAIGA_CLAVE` de GitHub. No duplica issues: si ya hay uno abierto, lo comenta. Mapeo GitHub → Taiga en `.taiga/miembros.json` (Ananagl, JuanDav-L, mariam7799). **Evidencia:** issue #85 creado y cerrado automáticamente.
- **Backlog:** `scripts/taiga/cargar_backlog.py` creó 14 historias (una por módulo de requisitos) y 70 tareas (una por requisito).
- **Scripts del repositorio:** `nueva_rama.sh`, `integrar.sh`, `liberar.sh` (crea la etiqueta vX.Y.Z) y `compilar_y_probar.sh`. Versión actual en producción: v1.0.2.
- **El CI sí encontró errores reales:** el nombre de la imagen de Docker debía ir en minúsculas, y en móvil faltaba que la tabla del comprobante fuera accesible con teclado. Ambos se corrigieron en la v1.0.2.

## 10. Métricas y monitoreo

- **Del equipo** (`scripts/metricas/metricas_taiga.py`, lee el Taiga real): avance del tablero (hoy 93 %), tareas e issues abiertos y cerrados por integrante, horas promedio de resolución, fallos de CI y errores de producción.
- **De la aplicación:** cada error 500 queda en `registro_error` con su correlación. Si el mismo error ocurre **5 veces en 10 minutos** en la misma ruta, el backend crea solo **un** issue "[Producción]" en Taiga. El ADMIN consulta `/api/v1/monitoreo/errores` y `/monitoreo/metricas` (ventas del día, anulaciones, productos sin existencias, valor del inventario, errores y accesos denegados).

## 11. Requisitos del enunciado y dónde se cumplen

| Requisito | Cómo se cumple / dónde mostrarlo |
|---|---|
| Java, Spring Boot, JPA, Lombok | Repositorio backend; entidades en `*/entidad`, repositorios JPA, Lombok en entidades y servicios |
| Validaciones | Bean Validation en DTO + reglas en servicios + `CHECK` en BD + Yup en el frontend |
| Seguridad y autenticación | Spring Security: sesión, Recordarme, CSRF, roles, bloqueo y auditoría (sección 5) |
| Javadoc, OpenAPI, JSON | `mvn javadoc:javadoc`; Swagger en el perfil dev; respuestas JSON y ProblemDetail |
| i18n | Backend: `mensajes.properties` es/en según `Accept-Language`. Frontend: selector de idioma |
| Encabezados de archivos | En todos los archivos; verificados por script en el CI |
| React, MUI, responsive, accesible | Tablas que pasan a tarjetas en móvil; axe sin violaciones; tema claro/oscuro |
| Tokens de diseño, Formik, Yup, JSDoc | `src/app/tema` (tokens), formularios con Formik + Yup, JSDoc en cada componente |
| Sesión persistente | "Recordarme" 7 días; la sesión se restaura al recargar |
| Pruebas (todos los tipos) | Sección 8; catálogo de 220 casos; informe en `docs/fase-5-pruebas` |
| Scripts de BD, compilación y repositorio | `db/*.sh`, `scripts/*.sh` del backend, `scripts/repositorio` y `compilar_y_probar.sh` |
| CI/CD: desplegar si pasa; si falla, issues, tareas y notificación en Taiga | Sección 9; evidencia: issue #85 y la historia "Fallos de CI" |
| Métricas y monitoreo en Taiga | Sección 10; `docs/fase-7-metricas` |
| Documentación por fases | Repositorio docs: fases 1 a 7, ADRs, informes por incremento, PDF de prompts |

## 12. Guion sugerido para la demo (10 minutos)

1. Abrir el sistema (1 minuto antes, por el plan gratis) → entrar como **admin** → cambiar de idioma y de tema.
2. **Ajuste** de inventario inicial (20 a $400) → **compra** (10 a $550) → mostrar el kardex con **CPP $450**.
3. **Venta** de 5 pagando $10.000 → cambio → **comprobante** con la leyenda de documento simulado → **devolución** de 2 desde la venta.
4. Entrar como **operativo**: no ve costos ni ajustes; si escribe la URL de auditoría ve "Acceso denegado".
5. **Auditoría** filtrada por usuario → **GitHub Actions** (pipeline en verde) → **Taiga** (issue #85 cerrado, tablero al 93 %).

**Preguntas probables:** "¿qué pasa si dos venden la última unidad a la vez?" → bloqueo FOR UPDATE + CHECK; probado con hilos reales. · "¿por qué no se borran documentos?" → trazabilidad: se anulan con movimiento compensatorio. · "¿cómo se calcula el costo?" → CPP ponderado con BigDecimal (ejemplo 1.300). · "¿qué pasa cuando falla una prueba?" → no se despliega y se crea un issue en Taiga asignado al autor del commit.